

Keeping Agents on a Leash: An Eight-Domain Control Model for AI Agents with Delegated Authority

Anton Dziatkovskii 

Platinum Software Development Company, Brisbane, Australia
dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054
<https://github.com/tonydzi> · <https://tonydzi.github.io>

September 2026

Abstract

An agent with delegated authority is an attack surface with credentials. The research answer is defenses evaluated on benchmarks; those benchmarks report that indirect prompt injection still succeeds between 10.7% and 29.6% against frontier models in production-like settings [14], and that of ten recent defenses on open-ended tasks almost all are either insufficiently secure or significantly over-defensive [13]. Operators must nevertheless ship. This paper presents LEASH-8, an eight-domain control model — identity, secrets, tool supply chain, approvals, containment, egress, observability, incident response — stated so that a small operation can implement it and, more importantly, *evidence* it: each domain carries a minimal implementation and an evidence test its owner must be able to satisfy in under a minute. Around it we give three instruments: a 24-statement scorecard with bands, a reference architecture in which the model plans, a deterministic gate authorizes and a scoped executor acts, and a checklist for the approval leg together with the designs that fail it. We claim no novelty at the level of principles: every domain restates least privilege, zero trust, capability security, the confused deputy, supply-chain integrity or operational monitoring, and for organizations with security teams the ground is already covered by enterprise maturity models. Section 3 gives a per-domain novelty ledger that says “no novelty” where that is the honest verdict. What we contribute is operationalization for the operator who has no security team: the smallest control set we could defend, the evidence question that makes each control checkable, and two open reference implementations. The patterns come from a five-machine production agent fleet, and we mark every such claim as either backed by a public artifact or as unaudited operator experience. This is a design paper, not an evaluation: we present no measurement that a higher score produces fewer incidents, and we say what would falsify us.

1 Introduction

The security question for an LLM agent is no longer whether the model can be talked into saying something. It is what the agent can *do* while it is being talked into it. An agent wired to a shell, a browser, a messenger, a payment method and a file system holds delegated authority; a successful injection there does not produce a bad sentence, it produces a transfer, a deletion, a message to five hundred people.

Two answers dominate. Vendors sell platforms. Researchers build defenses and evaluate them. The second is the more honest, and its results are sobering: on AgentDyn’s open-ended tasks, ten state-of-the-art defenses were found either not secure enough or over-defensive to the point of destroying utility [13], and LivePI, testing five frontier models across seven input surfaces and twelve attack families in a production-like harness, reports total attack success between 10.7% and 29.6% [14]. Both are successors to AgentDojo [12], the benchmark that made this class of evaluation standard. There is no filter that ends the problem.

There is, however, a third answer, older than the field and unglamorous: bound what a compromised component is allowed to do. That is not a defense in the benchmark sense — it does not stop the injection — and it is the only lever whose effect does not depend on out-guessing the attacker’s prose. This paper is about doing that deliberately rather than accidentally, in operations that have no security engineer to do it for them.

Contributions. (i) **LEASH-8**, an eight-domain control model for agents with delegated authority, each domain given as threat, minimal implementation, and an *evidence test* — the artifact its owner must be able to produce in under a minute (§4). (ii) A **plan/authorize/execute** reference architecture with an action-tier taxonomy and the anatomy of an out-of-band approval (§5), together with an **approval-design checklist** and a catalogue of common designs that fail it (§6). (iii) A **24-statement scorecard** usable in five minutes, with its design rationale and its measurement weaknesses stated as plainly as its bands (§7). (iv) An explicit **grounding table** (§8) separating what our production operation has published as an artifact from what remains our word, and (v) a **novelty ledger** (§3) naming, per domain, the prior art that already covers it.

What this paper is not. It is not an evaluation: no user study, no red-team competition, no comparison of scored against unscored systems, and no claim that these controls prevent compromise. The strongest claim we defend is the one the model is built around — they shrink blast radius, raise attacker cost, and make agent actions reviewable — and, unlike “our agents are secure,” each of those is something an operator can be asked to show.

2 The setting: what delegated authority buys the attacker

Three properties separate this setting from ordinary application security.

Every input is a potential instruction. Indirect prompt injection [10] turns web pages, issues, emails and tool descriptions into candidate commands, because instruction channel and data channel are the same token stream. This is the confused deputy [8] with a natural-language front end, and it is why authorization cannot live inside the component that reads untrusted text.

The supply chain now ships prose. Installing an agent skill installs code *and* instructions, and the model is trained to obey the second. In the largest incident so far, an audit of an agent skill marketplace found 341 malicious entries among 2,632 skills, a figure that grew to 824 as the marketplace expanded past 10,700 [28] [28].¹ The same year, registry worms turned single stolen publish tokens into hundreds of poisoned package versions, in one case committing agent hook files into source repositories so that merely opening a checkout could execute the payload [29] — an install path that never touches a package manager, aimed at packages sitting next to model credentials.

The blast radius is whatever you granted, times the time nobody looked. Agent actions are fast, unattended and numerous, and their characteristic operational failure is silence rather than error: the routine that ran and did nothing, the worker that died quietly, the action nobody logged [3].

Threat model, stated plainly. The adversary supplies text the agent will read — a web page, an issue, an email, a file, a tool or skill description — and may also publish a package or skill the operator installs. The adversary does not control the operator’s host, the gate’s code, or the approver’s account; if any of those falls, nothing in this paper helps. The trust boundary runs between the model, which we assume can be induced to propose anything, and the gate plus executor, which we assume execute only what code decided. Out of scope: attacks on the model weights, side channels, denial of service, and insider compromise of the approver. The design premise follows from that boundary: assume the model will occasionally be instructed by its inputs, and place the controls where that assumption is survivable.

3 Related work, and where we claim nothing

LEASH-8 is an assembly of other people’s principles. The honest way to present such a thing is a ledger (Table 1), stating for each domain what already covers it and what — if anything — we add.

¹Our own repository previously summarized this as “roughly one in five” marketplace skills compromised. The audited figure is 13% at first count and lower after the marketplace grew. This paper uses the source figures, and the repository text has been corrected to match.

Domain	Prior art that already covers it	Our delta
1 Identity	Least privilege and separation of privilege [7]; agent identifiers and activity logs [15]; verifiable delegation across agent protocols [16]; A2A agent cards [20].	No novelty. We restate it as a two-artifact evidence test: show two agents, show two credentials.
2 Secrets	Least privilege [7]; short-lived scoped credentials, standard practice; OAuth 2.1 for protected tool servers [21].	No novelty. One agent-specific framing: the always-loaded context is a publication surface, so the test is a grep of that context.
3 Supply chain	Software supply-chain integrity frameworks [25]; pinning and review, standard practice.	No novelty on mechanism. Emphasis only: review must cover natural-language instructions, not just code [28].
4 Approvals	Confused deputy [8]; capability security [9]; CaMeL enforces policy outside the model over extracted flows [11]; OpenPort specifies risk-gated human review, time-bounded approvals, impact binding and state revalidation [17]; governance practice [23].	No novelty , including in the properties we learned the hard way: [17] specifies state revalidation independently. What remains ours is narrower — which classes may reach a human at all (§6) and a dependency-free implementation [2].
5 Containment	Zero trust [24]; sandboxing and read-only defaults, standard practice; capability confinement [9].	No novelty. One fleet-specific rule: configuration propagates one way, so a compromised worker cannot rewrite the canon.
6 Egress	Exfiltration control and egress filtering, standard practice; data-flow capabilities in CaMeL [11].	No novelty. Framing: enumerate the doors first — most small operators cannot list theirs — then gate the last one before publication.
7 Observability	Site-reliability monitoring practice; visibility measures for agents [15]; our own control-plane casebook [3].	No novelty in principle; counters-not-narrative and silence-as-incident are inherited from [3].
8 Incident response	Standard incident-response practice; capability thresholds and staged autonomy in frontier safety policies [23].	No novelty. The one agent-specific instrument is an intermediate mode: keep drafting, stop sending.

Table 1: Novelty ledger. Six of eight domains contribute nothing new; the model’s value, if it has any, lies in assembly, minimality and evidence.

Three neighbours deserve their own paragraph, because they are close enough that a reader should know why this paper is not redundant with them.

Alignment by principles. Constitutional AI [22] governs behaviour through written principles applied in training and self-critique. It is orthogonal to this paper: it shapes what the model *wants*, while every domain here assumes the model may be induced to want the wrong thing and asks what it can then reach. A constitution is not an authorization mechanism, and treating it as one reproduces the confused deputy with extra steps.

Enterprise maturity models. OWASP’s agentic AI governance maturity work and the Cloud Security Alliance’s AI security maturity model and MAESTRO layer decomposition [26, 27] cover this ground for organizations, with adoption tiers, governance levels and per-layer threat modelling. They are more complete than LEASH-8, and they presuppose a security function, a program and a budget. LEASH-8 is sized for the operator who is one or two people, has none of those, and would otherwise implement nothing: eight domains, 24 statements, five minutes, MIT-licensed. A reader with a security team should use [26, 27] and treat this paper as a compact checklist for one agent system inside that program.

Protocol-level formalization. AgentRFC formalizes agent-protocol security as invariants and model-checks specifications against them [18]; threat models for MCP, A2A and neighbouring protocols are surveyed in [19]. That work verifies protocols; this paper equips an operator running agents on top of whatever protocols exist. The two meet at one point: the operator’s declaration of what an agent is and holds — for which we use a signature-capable A2A agent card [20] and deliberately declare less than we could, since a card that overstates capability is a false claim with your name on it.

4 LEASH-8: the model

LEASH-8 is *layered enforcement, auditability and supply-chain hygiene* across eight domains. It is implementation-agnostic by construction: it names controls, not products. Each domain is specified by three things — why it earns its place, the minimal implementation that counts as having it, and the *evidence test*: the artifact its owner should be able to produce, in under a minute, when asked to prove the control exists (Table 2).

#	Domain	Minimal implementation	Evidence test (under one minute)
1	Identity	One principal per agent role: its own account, token or session, never a human’s superuser context; per-machine identity; author stamped into every action record.	Show two agents and two distinct credentials; show a log line naming agent, machine and task.
2	Secrets	Secrets in a store queried at use time, never in system prompt or persistent memory; scopes matched to the job; rotation without redeploying the fleet.	Grep the agent’s loaded context for key-shaped strings. It comes back empty.
3	Supply chain	Allowlist or internal registry; pinned versions; read-before-run review covering instructions as well as code; no production install-from-URL.	Show the allowlist, a pinned version, and one rejected candidate with the reason.
4	Approvals	Irreversible classes enumerated in writing; a deterministic gate between model and executor; out-of-band human approval with an allowlisted approver identity for the top tier.	Fire a gated action in a test; show it held; show the deny path working.
5	Containment	Read-only by default, writes granted per task; risky execution in a sandbox, worktree or container; configuration propagates one way to workers.	Show a worker machine failing, by design, to modify shared configuration.
6	Egress	A written, deliberately short list of channels through which data can leave; destination allowlists where the platform permits; a leak scan as a hard gate before external publication.	Show the list; show one publish attempt blocked by the gate.
7	Observability	Append-only action log outside the agent’s write scope; result counters per routine; watchdogs for which silence itself is an alarm.	Take yesterday’s run: show its counters, and show that the agent could not have rewritten the log.
8	Incident response	Kill switch per agent and per machine; an autonomy downgrade that keeps drafting and stops sending; a measured time-to-disable; a drill on the calendar.	Show the last drill’s timestamp and its time-to-disable number.

Table 2: The eight domains. Nothing in columns two and three is novel (Table 1); the last column is what makes it checkable.

The evidence test is the design decision that matters here. Security self-assessment degrades into intention: an operator who means to rotate credentials scores like one who does. Forcing every statement to terminate in a showable artifact — a log line naming its author, a grep that comes back empty, a blocked publish attempt, a drill timestamp — converts intention into a fact that either exists or does not [3].

Two domains are routinely misjudged, and both misjudgments are cheap to correct. *Secrets* are usually believed handled because a store exists, while the agent’s always-loaded context — system prompt, memory index, the configuration it re-reads every turn — is a separate surface no store protects; a credential the model can quote is one exfiltration prompt away from leaving. *Egress* is usually believed narrow, and is not: every messenger, mail account, webhook, git remote and outbound HTTP path is an exit, and the first version of the list is always longer than its author expected.

For a small operation we suggest the order approvals, egress, supply chain, observability — that is where the cheapest control removes the most authority — with identity and secrets usually half-done already. This ordering is judgment, not a finding (Table 4).

5 The pattern: the model plans, a gate authorizes, an executor acts

Domains 4 and 5 rest on one architectural decision, which we consider the highest-leverage one available in an agent system with delegated authority.

The anti-pattern is a single component that reads untrusted input and holds the authority to act: model in, action out. Under that architecture every injection success reported in [14, 13] is a live action.

The pattern separates three things that are usually fused:

1. **The model proposes and never holds the keys.** Its output is a structured request (“send *X* to *Y*”), not the act itself.
2. **A deterministic gate classifies and decides.** Code, not a model judging its own request: action classes,

allowlists, thresholds. It cannot be sweet-talked, and it is what CaMeL formalizes as policy enforcement over extracted flows [11].

3. **The executor is dumb and scoped.** It performs exactly the approved request with the minimal credential for that class — a payment executor cannot delete a repository.

Action tiers. Tier 0, reversible and internal (read, draft, query): execute and log. Tier 1, bounded and reversible-ish (move a file with a backup, post to an internal channel): execute, log with counters, reviewable after. Tier 2, irreversible or outward-facing (money, outbound messages to humans, deletion, publication, credential use): hold until out-of-band human approval. The rows matter less than two properties: the list exists in writing, and **undeclared classes default to the highest tier**, never the lowest.

Out-of-band approval. The approval must travel on a channel the attacker’s text cannot reach, and three properties do the work. *Identity authorizes*: a reply decides only if its sender identifier matches an allowlisted approver, so “Alex approved this” — in a document, an issue, a forward, a screenshot, a relaying bot — authorizes nothing, since none of those produces a reply from Alex’s account. *The token binds intent to a request*: an explicit token tied to a request identifier separates deciding from chatter that happens to contain the word “ok.” *Content the agent read is never approval*: text in any input is data, permanently. To these we add a fourth, learned the hard way and discussed in §6: an approval expires as a picture of the world even while it remains valid as permission.

6 Designing the approval leg, and the designs that fail

The plumbing of an approval gate is easy; the design around it is where operations lose. Our checklist [1] covers enumeration, gate placement, the approval channel, time and state, human factors and evidence. Rather than reproduce it, we give the failure catalogue, since a checklist is best read as the negation of designs that have failed (Table 3).

Design	Why it fails
“The model asks itself whether this is safe before acting.”	The judge is the suspect: same context, same injectable input.
“Approval is the user saying ok in the chat the agent also reads documents in.”	Injected text is indistinguishable from consent on that channel.
“One yes at the start of the session covers what follows.”	Unbounded in scope and time; the classic grant that is never withdrawn.
“If no answer in ten minutes, proceed.”	Timeout-as-consent inverts the default: silence must never authorize.
“Approvals live in the agent’s own memory or log file.”	The suspect keeps the evidence.
“Everything the agent is unsure about goes to the human.”	Alarm fatigue by construction; see below.

Table 3: Approval designs that fail the checklist, with the property each violates.

The last row is the one we did not anticipate, and the most useful thing we can pass on. Our first gate asked about everything the agent was unsure of; within a week the channel carried a file rename, a documentation push, a cache reindex — and, at 09:40, a payment. Nobody read the payment, because by day four the channel had taught its reader that nothing in it needed reading. *A gate that asks about everything is not a safety mechanism; it is a way of moving responsibility onto someone who has stopped looking.* The repair is a decision-class table in which most classes are explicitly the agent’s own to decide and journal, and in which the gate *refuses* to queue them — so an agent cannot talk itself into a slot in a human’s attention — while only two classes reach a person: actions needing human hands (a second factor, a physical confirmation) and the irreversible or outward-facing ones. Our open implementation ships that table, the silence ladder (wait, re-ping, escalate, give up permanently,

and silently retire anything a downed supervisor never asked), and a usage counter, in one standard-library file with 36 tests [2].

Two further properties belong to design rather than plumbing. **Silence must be a named state**, not an absence: every hold notifies someone, every deny is loud, and a gate that silently swallows a request is worse than no gate, because the work simply disappears. And **a stale approval must be re-read against the world**: we watched an agent faithfully execute a six-hour-old approval for work completed in the interim — the permission was valid, the picture was stale. We reached this the expensive way; OpenPort specifies the same property as an optional state-witness profile that revalidates preconditions and fails closed on mismatch [17], which is the better statement of it.

Finally, the uncomfortable half. An approval queue is a queue to one person, who does not scale, does not run at 03:00, and gets tired. A human in the *middle* of a pipeline is an architecture bug; a human at its *ends* — setting the goal, accepting the result — is the design. This is why a counter ships with the gate rather than as an afterthought: for one stretch of this year our own ask queue grew faster than it was read, which is a gate working perfectly and producing nothing, because *delivered to a human* is not *decided by a human*.

7 The scorecard as an instrument

The scorecard [1] is 24 statements, three per domain, each scored 0 (not true), 1 (partially true) or 2 (fully true *and* the evidence could be shown in under a minute), for a maximum of 48, with named bands: 0–16 free-running, 17–32 loose leash, 33–42 on a leash, 43–48 operator-grade.

Three design choices are deliberate: the three-point scale exists so that “we have something” cannot masquerade as “we have this”; the evidence clause separates 1 from 2 and is the entire reason this is not a survey of intentions; and the statement count is capped so that the exercise finishes, since an assessment the operator abandons scores zero on every domain in practice.

We are equally explicit about what it is not. It is **self-reported**: nobody audits the score, and it is trivially gamed by anyone who wants a number rather than a decision. It has **no inter-rater reliability data**: we have not had several assessors score one system and compared them, so its variance is unknown. Its **band thresholds are judgment, not data** — chosen so that a system with a zero in identity, approvals or egress cannot reach the top bands, and nothing more principled than that. It measures **control coverage, not security**: a high score means smaller blast radius and reviewable actions, and an operation can hold one while being compromised through a domain nobody thought to include. And crucially, **we have no evidence that a higher score yields fewer or cheaper incidents**; establishing that would require exactly the study this paper does not contain.

We publish it nonetheless: in a small operation the alternative to an imperfect instrument is not a better one, it is nothing — and the most common outcome of a first honest scoring is a zero in a domain its owner had never considered, most often egress.

8 Grounding: what is an artifact and what is our word

The patterns here come from a multi-machine agent operation — five machines across Windows, macOS and a cloud anchor, running scheduled agent routines daily since mid-2026, described as an experience report in a companion preprint [4]. Experience is not evidence unless it can be pointed at, so Table 4 states, claim by claim, whether a public artifact backs it or whether the reader is taking our word.

Claim used in this paper	Public artifact	Status
Decision classes; only two may interrupt a human; the gate refuses to queue the rest	<code>agent-approval-gate</code> [2], 36 stdlib tests	Artifact, runnable
Identity authorizes, token binds intent; “X approved this” in content authorizes nothing	<code>agent-approval-gate</code> [2]	Artifact, runnable
Silence ladder: wait, re-ping, escalate, give up, retire a backlog silently	<code>agent-approval-gate</code> [2]	Artifact, runnable
Ask queue grew faster than it was read; six-hour-old approval executed against a changed world	narrative and counter tooling in [2]; underlying logs not published	Experience, no raw data
Silent failure is characteristic; counters over narrative; silence as incident	control-plane casebook [3], 11 dated cases	Artifact (partly reproducible)
Eight domains, minimal implementations, evidence tests, scorecard, checklist	<code>agent-leash</code> [1]	Artifact (the model itself)
One-way configuration propagation; workers cannot rewrite the canon	pattern stated in [1]; our implementation unpublished	Experience, no artifact
Leak scan as a hard gate before external publication	tooling exists [6]; rule set unpublished	Experience, partial
Per-agent kill switch, autonomy downgrade, time-to-disable	none published; no drill number released	Experience, unmeasured
Implementation order (approvals, egress, supply chain, observability) pays fastest	none	Opinion

Table 4: Grounding. Rows marked experience are operator judgment; treat them as hypotheses, not findings.

We do not publish our live control surfaces — topology, channels, approver identities, rule sets — and this limits verifiability in a real rather than rhetorical way: a reader can run our reference implementations, but cannot audit the operation the patterns were abstracted from. We state the trade so that nobody mistakes the repositories for a disclosure of the system.

9 Limitations, and what would falsify this

An assembly can be wrong in three ways: the domains may not be exhaustive, the minimal implementations may not be minimal, and the instrument may not measure what it claims. On the first we are openly exposed — eight domains fell out of one operation’s experience plus published guidance, and a domain absent from the list is invisible to the score; multi-agent trust relationships and cross-session memory poisoning are the two candidates we most expect to need a place. On the second, “minimal” is our judgment, untested against any cost baseline. On the third, §7 states the measurement weaknesses without softening them.

The grounding is $N=1$, unaudited from outside, over a few months, and several claims in Table 4 are marked as our word precisely because they cannot currently be checked. Nor is any of this an outcome claim: we have not shown that an operator scoring 40 is attacked less often, or less expensively, than one scoring 20.

We would consider the paper usefully falsified by any of the following, each of which we would rather see than agreement: a small operation scoring highly on this instrument and being compromised through a domain the model does not contain; a study finding that scores do not distinguish operations by incident cost; two assessors scoring one system and diverging widely; or a demonstration that the plan/authorize/execute separation is routinely bypassed in real deployments through the gate’s own interface rather than through the model. The second and third are cheap experiments that we cannot run alone, and we will help anyone who wants to.

Code, data and contact

The model, scorecard, checklist and reference agent card are a live, open repository: <https://github.com/tonydzi/agent-leash> (MIT). The approval gate is a separate standard-library implementation with tests, transports and a stated threat model: <https://github.com/tonydzi/agent-approval-gate> (MIT). The control-plane casebook grounding domain 7 is at <https://github.com/tonydzi/agent-c>

ontrol-plane-casebook. Related open components from the same operation — multi-machine consensus (claw-consensus), fleet deployment with landing proof (fleet-deploy), an independent review panel gating “done” (secondop-panel), publication tooling that ships internals without leaking them (oss-publish), governance rules as files (claude-bible) — are all under <https://github.com/tonydzi>.

Corrections are the contribution we value most: open an issue on agent-leash. Author: Anton Dziatkovskii — dzyatkovskiy.a2@gmail.com · ORCID 0000-0001-7408-3054 · <https://github.com/tonydzi> · <https://tonydzi.github.io>. Companion preprints: [4, 5].

Authorship, responsibility, independence. This paper and the accompanying repositories are drafted by an AI agent operating as the author’s synthetic co-founder, under the author’s editorial control; the author is the responsible human for every claim made here, every commit carries provenance trailers naming tool, model, machine and operator, and corrections are logged publicly. We are not a security vendor, and this work is not affiliated with, endorsed by, or reviewed by any vendor named in it.

References

- [1] A. Dziatkovskii. agent-leash: LEASH-8, an eight-domain control model for AI agents with delegated authority. <https://github.com/tonydzi/agent-leash>, 2026. MIT license.
- [2] A. Dziatkovskii. agent-approval-gate: out-of-band human approval for unattended agents, with decision classes. <https://github.com/tonydzi/agent-approval-gate>, 2026. MIT license.
- [3] A. Dziatkovskii. Silent by construction: a casebook of control-plane failures in a production LLM agent fleet. Preprint and repository, <https://github.com/tonydzi/agent-control-plane-casebook>, 2026.
- [4] A. Dziatkovskii. Operating a human-governed multi-machine LLM agent fleet: an experience report. Preprint, 2026. doi:10.5281/zenodo.22639714.
- [5] A. Dziatkovskii. Companion preprints, 2026: Homeostatic governance — a Gorsky–Anokhin alternative to the Viable System Model, doi:10.5281/zenodo.22639712; Le Chatelier tokenomics, doi:10.5281/zenodo.22639710; When does graph expansion help personal-corpus retrieval?, doi:10.5281/zenodo.22639718.
- [6] A. Dziatkovskii. oss-publish: publishing internal tooling without leaking the operation. <https://github.com/tonydzi/oss-publish>, 2026.
- [7] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proc. IEEE*, 63(9):1278–1308, 1975.
- [8] N. Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [9] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM AISec*, 2023. arXiv:2302.12173.
- [11] E. Debenedetti et al. Defeating prompt injections by design. arXiv:2503.18813, 2025.
- [12] E. Debenedetti et al. AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks*, 2024. arXiv:2406.13352.
- [13] AgentDyn: a dynamic open-ended benchmark for evaluating prompt injection attacks of real-world agent security systems. arXiv:2602.03117, 2026.
- [14] L. Zhao, A. Bhaskar, and E. Dobriban. LivePI: more realistic benchmarking of agents against indirect prompt injection. arXiv:2605.17986, 2026.
- [15] A. Chan et al. Visibility into AI agents. In *ACM FAccT*, 2024. arXiv:2401.13138.
- [16] AIP: agent identity protocol for verifiable delegation across MCP and A2A. arXiv:2603.24775, 2026.
- [17] G. Zhu, C. Wang, Z. Wang, Z. Li, and Q. Li. OpenPort protocol: a security governance specification for AI agent tool access. arXiv:2602.20196, 2026.
- [18] S. Zheng and Q. Zhang. AgentRFC: security design principles and conformance testing for agent protocols. arXiv:2603.23801, 2026.
- [19] Security threat modeling for emerging AI-agent protocols: a comparative analysis of MCP, A2A, Agora and ANP. arXiv:2602.11327, 2026.
- [20] Agent2Agent (A2A) protocol specification, v1.0. Linux Foundation, 2026. <https://a2a-protocol.org>.
- [21] Model Context Protocol: authorization specification (OAuth 2.1 with PKCE for protected servers), 2025–2026. <https://modelcontextprotocol.io>.
- [22] Y. Bai et al. Constitutional AI: harmlessness from AI feedback. arXiv:2212.08073, 2022.
- [23] Governance and staged-autonomy practice: OpenAI, Practices for governing agentic AI systems, 2023; Anthropic, Responsible scaling policy (AI safety levels), 2023–2026; OpenAI Preparedness Framework; Google DeepMind Frontier Safety Framework.
- [24] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. Zero trust architecture. NIST Special Publication 800-207, 2020.
- [25] Supply-chain Levels for Software Artifacts (SLSA). OpenSSF. <https://slsa.dev>.
- [26] OWASP GenAI Security Project. State of agentic AI security and governance, including the agentic AI governance maturity model, 2026.
- [27] Cloud Security Alliance. AI security maturity model; MAESTRO threat modeling for agentic AI, 2025–2026.
- [28] Koi Security, ClawHavoc: malicious skills in an agent skill marketplace, 2026, <https://www.koi.ai/blog/clawhavoc-341-malicious-clawedbot-skills-found-by-the-bot-the-y-were-targeting>; and Unit 42, Palo Alto Networks, Agent skill marketplaces and the emerging AI supply chain threat, 2026, <https://unit42.paloaltonetworks.com/openclaw-ai-supply-chain-risk/>.
- [29] Analyses of the 2026 npm registry worms and maintainer-token compromises affecting AI-agent packages: Unit 42; Microsoft Security; Wiz, 2026.